

5. 排序与打分规则

排序不是单一向量分数。OpenViking 在不同阶段会使用向量分数、rerank 分数、目录父级分数传播、阈值过滤和热度分数融合，最终返回 MatchedContext.score。

局部候选分数：

```
local_score = rerank_score if rerank 可用 且 mode=thinking else vector_score
```

递归传播分数：

```
final_score = alpha * local_score + (1 - alpha) * parent_score  
alpha = SCORE_PROPAGATION_ALPHA = 0.5
```

阈值过滤：

```
final_score > score_threshold  
或 score_gte=true 时 final_score >= score_threshold
```

最终展示分数：

```
display_score = (1 - HOTNESS_ALPHA) * semantic_score + HOTNESS_ALPHA * hotness_score  
HOTNESS_ALPHA = 0.2
```

排序因子	来源	作用
vector_score	向量库 dense/sparse/hybrid 搜索返回的 _score	基础相似度，用于全局召回和子节点召回。
rerank_score	RerankClient.rerank_batch(query, documents)	在 thinking 模式下重排候选摘要，提高语义贴合度。
parent_score	当前进入目录的分数	通过分数传播让高相关目录下的子节点获得合理加权。
score_threshold	调用参数或 rerank_config.threshold	过滤低分候选，控制噪声。
hotness_score	active_count + updated_at	常用、近期更新的上下文获得轻微提升。
dedup by URI	collected_by_uri	同一 URI 多次命中时保留最高 final_score。
relations	VikingFS relation graph	最终结果带最多 5 个相关节点摘要，扩展上下文。

Rerank 失败时怎么办

如果 rerank 服务异常、返回长度不一致或未配置，系统会记录 warning，并回退到向量分数。这样检索质量可能下降，但服务不会因为重排失败而不可用。

为什么要分层排序	解释
避免只命中孤立碎片	L0/L1 先定位相关知识区域，再进入子节点，能保留文档结构和上下文关系。
降低大库搜索成本	全局召回只选起点，递归搜索只展开高分目录，减少无关节点遍历。
改善答案可解释性	MatchedContext 保留 level、uri、abstract、relations，方便追溯来